# Sthyra CRM — Phase 0 Status Report

Phase 0 Foundation: Complete Status Report and Execution Roadmap

Engineering Team · Sthyra CRM

August 2026

Executive Summary

Table of Contents

# Executive Summary

Sthyra CRM is a multi-tenant, visual-intelligence platform for the construction industry. The full product plan — 18 months, ~$8.4M Year-1 budget, 13 products, multi-region cloud, FedRAMP Moderate — was produced in a single planning session in which **10 specialist "engineer" agents** were dispatched in parallel to design the system end-to-end. The plan was synthesized into a single 461-line master document and a 449-line technical appendix (the "Sthyra CRM Master Plan") saved to the operator's plan store.

This document reports the **Phase 0 Foundation** that was actually built and committed. Phase 0 is the 3-month "Foundry" tranche from the master plan: monorepo scaffold, three tenant-scoped services, design system, observability, auth middleware, dashboard shell, CI/CD, and Docker. Everything in Phase 0 **runs**, is **tested**, and is **boot-verified end-to-end**.

Three commits sit on `main`. **114 tests pass across 8 packages**. Five services boot and respond to curl with RFC 7807 problem+json + request-id propagation. The architecture reflects every pinned decision from the master plan: teal+amber palette (Appendix L.1), strict TypeScript, REST + RFC 7807 + Idempotency-Key, Repository pattern, tenant-scoped region on every record.

The "Detailed Steps To Be Done" section (§10) covers the **Phase 1+ backlog** — the seven remaining product tracks from the master plan — with exact bite-sized tasks, file paths, code shape, test commands, and verification steps. This document is the handoff packet for the next engineer or AI coding agent.

# Table of Contents

# 1. The Sthyra CRM Plan — what was synthesized

The master plan was produced by **10 specialist agents** dispatched in parallel and given the verified OpenSpace AI product line (Capture → Coordinate → Act; the 5-product split Capture / Field / Track / Air / BIM+; FedRAMP Moderate; SOC 2 Type II; 69 B sq ft captured; 131 countries; 100K+ projects — pulled from `openspace.ai` via `curl` since web-search keys weren't configured in the operator's environment).

## 1.1 Agent dispatches

| Wave | Agent | Output |
|---|---|---|
| 1 | Product Lead | PRD for the 13-product line, 8 personas, pricing tiers |
| 1 | Tech Lead / Staff Engineer | Service catalog, stack, ADR set |
| 1 | Backend | REST + GraphQL + gRPC API surface, ingestion pipeline, repos |
| 2 | Frontend Web | Next.js 14 design system, 12-page IA, 360 viewer spec |
| 2 | Mobile (iOS + Android) | KMM + native shells, capture workflow, sync engine |
| 2 | CV/3D + AI/ML | Spatial AI pipeline, Copilot, progress tracking, MLOps |
| 3 | Design/UX | Brand identity (originally "STRATUM"), motion system, UX research |

| Wave | Agent | Output |
|---|---|---|
| 3 | DevOps/SRE | Multi-region cloud, CI/CD, observability, SLOs |
| 3 | Security & Compliance | Zero-trust, identity, data protection, FedRAMP roadmap |
| 4 | QA / Test Engineering | Test pyramid, release gates, beta program |

Each agent was briefed as a real engineer joining the team. Outputs were synthesized into two documents saved to `~/.hermes/plans/`:

- `2026-08-08_090307-sthyra-crm-visual-intelligence-platform.md` — master plan, 461 lines, 18 sections
- `2026-08-08_090307-sthyra-crm-visual-intelligence-platform-APPENDIX.md` — technical appendix, 449 lines, including a process log of cross-agent conflicts that surfaced and were resolved

## 1.2 What the plan specifies

A condensed view of the master plan's scope:

- **13 products** — Capture, Field, Track, Air, Model, Copilot, Voice, Live, Twin, ESG, Claims, Edge, Admin & Trust
- **Pricing model** — Free / Pro ($480/seat/mo) / Enterprise / Gov (FedRAMP)
- **Architecture** — multi-region active/active, three planes (edge, control, data); Postgres + TimescaleDB + Redis + ClickHouse + OpenSearch + S3; SPIFFE/SPIRE for workload identity; Elixir/Phoenix Channels for realtime
- **AI/ML pipeline** — PyTorch + Triton + KServe; COLMAP/GLOMAP SfM; OpenMVS dense; 3D Gaussian Splatting novel-view; SAM-2 + DINOv2 + Grounding DINO; Llama 3 Copilot with RAG + voice I/O + safety pipeline
- **Mobile** — Swift/SwiftUI + Kotlin/Compose shells, KMM shared core, on-device Whisper, ARKit/ARCore, AR overlay of BIM
- **Compliance** — SOC 2 Type II day-one, FedRAMP Moderate by GA, ISO 27001, HIPAA, CMMC L3, regional data residency (US/EU/UK/AU/JP/KSA)
- **18-month roadmap** — Phase 0 (Foundry), Phase 1 (MVP), Phase 2 (Beta), Phase 3 (GA), Phase 4 (differentiators)
- **Team & cost** — 12 founding hires, $8.4M Year-1, $18M Year-2 to GA

The complete plan lives at `~/.hermes/plans/`. This document focuses only on what was actually built.

# 2. Phase 0 Foundation — what was built

Phase 0 spanned **three commits on `main`**. Each commit added real, tested, running code — no stubs, no placeholders.

## 2.1 Commit timeline

```
a0d78aa  feat(phase-0): monorepo scaffold, design tokens,
         org-service HTTP MVP, lint config
         ── design system, observability package, three services, HTTP
layer,
             ESLint + Prettier, GitHub Actions CI

fad7a02  feat(phase-0): postgres repo, project-service, user-service,
         observability, dashboard, CI
         ── Postgres-backed repository, project + user services,
Next.js
             dashboard shell, Docker Compose, three-stage CI

51526f4  feat(phase-0+): membership service, auth middleware,
         dashboard live data, list endpoint
         ── shared auth package, membership service for RBAC,
dashboard
             wired to live services, GET /v1/orgs endpoint
```

## 2.2 What ships

| Package / Service | Purpose | Tests |
|---|---|---|
| `@sthyra-crm/tokens` | Design system: 3 modes (light/dark/high-contrast), teal+amber palette, type, | 11 |

| Package / Service | Purpose | Tests |
|---|---|---|
| | motion, radii, shadows, CSS variable export | |
| `@sthyra-crm/observability` | Structured JSON logging + AsyncLocalStorage request-id + Fastify plugin | 5 |
| `@sthyra-crm/auth` | Shared bearer-token middleware (Fastify plugin) verifying via user-service `/v1/tokens/verify`; attaches `req.principal`; RFC 7807 401/503 | 7 |
| `@sthyra-crm/org-service` | Tenant-scoped orgs. REST + Postgres + in-memory. POST/GET/list. Idempotency-Key. | 26 |
| `@sthyra-crm/project-service` | Tenant-scoped projects belonging to an org. Archive FSM. Postgres + in-memory. | 25 |
| `@sthyra-crm/user-service` | Identity, RBAC, opaque SHA-256 hashed tokens. Bearer-token verify endpoint. | 15 |
| `@sthyra-crm/membership-service` | User↔org and user↔project bindings. 5 org roles + 5 project roles. Auth wired. | 19 |
| `@sthyra-crm/dashboard` | Next.js 14 App Router shell. Home page (live org/project rollups). `/orgs/new` form. `/orgs/[orgId]/projects` drill-down. `/immersive` marketing placeholder. | 6 |
| **Total** | | **114** |

Files: 42 TypeScript files in `src/`, 70 total source files (incl. JSON, YAML, CSS, markdown), **~4,844 LOC** (TypeScript/TSX only, excluding dist).

## 2.3 Local infrastructure

| File | Purpose |
|---|---|
| `docker-compose.yml` | Postgres 16 with healthcheck, persistent volume, optional pgAdmin profile |
| `.env.example` | DATABASE_URL template, PORT, HOST |
| `.github/workflows/ci.yml` | Three-stage CI: unit-tests · integration-tests (Postgres) · quality-gates (typecheck + build + audit) |
| `.eslintrc.json` + `.prettierrc.json` | Strict ESLint 9 + Prettier 3, format-on-save |
| `README.md` | Architecture doc + working agreement for AI coding agents |
| `pnpm-workspace.yaml` | Three workspace roots: `packages/*`, `services/*`, `apps/*` |

# 3. Repository Layout

```
sthyra-crm/
├── packages/                          # reusable library code
│   ├── tokens/                        # @sthyra-crm/tokens — design system
│   │   ├── src/index.ts               # brand colors, semantic tokens, 3 modes
│   │   ├── src/index.test.ts          # 11 tests
│   │   └── tsconfig.json
│   ├── observability/                 # @sthyra-crm/observability — logging + request-id
│   │   ├── src/index.ts               # AsyncLocalStorage + Fastify plugin
│   │   ├── src/index.test.ts          # 5 tests
```

```
│   │   └── tsconfig.json
│   └── auth/                          # @sthyra-crm/auth — bearer-
token middleware
│       ├── src/index.ts               # installAuthPlugin() with
verify seam
│       ├── src/index.test.ts          # 7 tests (public/health, 401
paths, etc.)
│       └── tsconfig.json
│
├── services/                          # stateful backend services
│   ├── org-service/                   # @sthyra-crm/org-service
│   │   ├── src/index.ts               # OrgService + Repository
interface
│   │   ├── src/postgres-repo.ts       # PostgresOrgRepository
│   │   ├── src/http.ts                # Fastify server (RFC 7807,
Idempotency-Key)
│   │   ├── src/cli.ts                 # in-memory CLI
│   │   ├── src/postgres-cli.ts        # Postgres CLI
│   │   └── src/*.test.ts              # 26 tests
│   ├── project-service/               # @sthyra-crm/project-service
│   │   ├── src/index.ts               # ProjectService + archive FSM
│   │   ├── src/postgres-repo.ts       # PostgresProjectRepository
│   │   ├── src/http.ts                # Fastify server
│   │   ├── src/cli.ts
│   │   ├── src/postgres-cli.ts
│   │   └── src/*.test.ts              # 25 tests
│   ├── user-service/                  # @sthyra-crm/user-service
│   │   ├── src/index.ts               # UserService + opaque token
store
│   │   ├── src/http.ts                # Fastify + bearer-token
verify endpoint
│   │   ├── src/cli.ts
│   │   └── src/*.test.ts              # 15 tests
│   └── membership-service/            # @sthyra-crm/membership-
service
│       ├── src/index.ts               # OrgMembership +
ProjectMembership
│       ├── src/http.ts                # Fastify with auth wired
│       ├── src/cli.ts
│       └── src/*.test.ts              # 19 tests
```

```
│
├── apps/                                  # end-user surfaces
│   └── dashboard/                         # @sthyra-crm/dashboard
│       ├── next.config.mjs
│       ├── src/app/layout.tsx             # emits CSS vars from tokens
│       ├── src/app/globals.css            # design-token-driven styles
│       ├── src/app/page.tsx               # home — live org/project
rollups
│       ├── src/app/immersive/page.tsx     # marketing placeholder
│       ├── src/app/orgs/new/page.tsx      # create-org form
│       ├── src/app/orgs/[orgId]/projects/page.tsx   # drill-down
│       ├── src/app/orgs/[orgId]/projects/archive-button.tsx
│       └── src/lib/api.ts                 # server-side fetch +
request-id
│
├── docker-compose.yml                     # Postgres 16 + pgAdmin
├── .github/workflows/ci.yml               # 3-stage CI
├── .eslintrc.json, .prettierrc.json
├── package.json                           # pnpm workspace root
├── pnpm-workspace.yaml
├── README.md                              # architecture + working
agreement
└── tsconfig.base.json                     # strict TS settings
```

# 4. Architectural Decisions (Pinned)

These decisions are recorded in the working agreement (§7). They are **not** to be re-decided without an explicit conversation.

## 4.1 Tenancy is data-modeled

Every record carries a `region` field. `(name, region)` is unique — same name in different regions is allowed. This is the foundation of data-residency compliance (FedRAMP Moderate, EU GDPR, etc.).

```ts
// From services/org-service/src/index.ts
export interface Org {
  readonly id: string;
```

```
  readonly name: string;
  readonly region: Region;          // 'us-east' | 'us-west' | 'us-
        fedramp' | ...
  readonly plan: Plan;              // 'free' | 'pro' | 'enterprise' |
        'gov'
  readonly createdAt: Date;
}
```

## 4.2 Repository pattern

Each service defines a `Repository` interface; `InMemory*Repository` for tests/dev, `Postgres*Repository` for prod. Call sites never touch the DB.

```
// Repository contract — service code consumes this
export interface OrgRepository {
  insert(org: Org): Promise<void>;
  findById(id: string): Promise<Org | null>;
  findByNameAndRegion(name: string, region: Region): Promise<Org |
        null>;
  list(query?: { region?: Region; limit?: number }): Promise<Org[]>;
}
```

The `PgClient` interface is the **only seam** for Postgres. Tests pass a `FakePgClient` (in-memory, ~50ms test suite); production uses the real `pg.Pool`. This is the standard “test against a contract, not a database” pattern.

## 4.3 Postgres-backed, parameterized queries only

No string concatenation. Every value flows through `$1, $2, ...` placeholders. A typed `UniqueViolationError` is thrown on duplicate-key violations. Migrations are idempotent (`CREATE TABLE IF NOT EXISTS`).

```
// From services/org-service/src/postgres-repo.ts
async insert(org: Org): Promise<void> {
  try {
    await this.opts.client.query(
      `INSERT INTO orgs (id, name, region, plan, created_at)
       VALUES ($1, $2, $3, $4, $5)`,
      [org.id, org.name, org.region, org.plan, org.createdAt],
    );
  } catch (err) {
```

```
    const e = err as { code?: string };
    if (e?.code === '23505') {
      throw new UniqueViolationError(
        `Organization "${org.name}" already exists in region
        "${org.region}".`,
      );
    }
    throw err;
  }
}
```

## 4.4 REST + RFC 7807 + Idempotency-Key

Every backend service exposes REST. Every error response is `application/problem+json` with `type`, `title`, `status`, `detail`, `instance`, `trace_id`, and `code`. Mutating endpoints honor the `Idempotency-Key` header.

```
$ curl -X POST http://localhost:8080/v1/orgs \
       -H "content-type: application/json" \
       -H "idempotency-key: req-001" \
       -d '{"name":"Hudson Tower GC","region":"us-east","plan":"pro"}'
{"id":"org_00000001","name":"Hudson Tower GC","region":"us-
        east","plan":"pro","createdAt":"..."}

$ # duplicate → 409 problem+json
$ curl ... -H "idempotency-key: req-002" ...
{"type":"https://sthyra-crm.dev/errors/conflict",
 "title":"Organization already exists",
 "status":409,
 "detail":"An organization named \"Hudson Tower GC\" already exists in
        region \"us-east\".",
 "trace_id":"d450b3b5-3521-49db-8328-bd3f35456ccc",
 "code":"already_exists"}
```

## 4.5 Request-ID propagation

Every request gets an `x-request-id` (incoming header or freshly minted). The same ID appears in:

- every JSON log line emitted during the request
- every RFC 7807 problem+json response (`trace_id`)

Implementation uses Node.js `AsyncLocalStorage` so log lines from any depth of the call stack carry the right ID. The observability package ships a Fastify plugin (`installRequestIdPlugin`) that wires the AsyncLocalStorage context per-request.

```
// From packages/observability/src/index.ts
app.addHook('onRequest', (req, reply, done) => {
  const id = req.headers['x-request-id'] ?? randomUUID();
  void reply.header('x-request-id', id);
  requestIdStore.run(id, () => done());
});

app.addHook('onResponse', (req, reply) => {
  emit('info', 'http_request', {
    method: req.method,
    url: req.url,
    status: reply.statusCode,
    duration_ms: Math.round(reply.elapsedTime ?? 0),
  });
});
```

Live log output, captured during smoke testing:

```
{"ts":"2026-08-08T05:18:05.164Z","level":"info","service":"sthyra-crm-
        service",
 "request_id":"17993628-1193-4f64-8ce0-
        7ac9c1141364","msg":"http_request",
 "fields":
        {"method":"GET","url":"/v1/health","status":200,"duration_ms":2}}
```

## 4.6 Bearer-token auth via @sthyra-crm/auth

A shared Fastify plugin that every backend service installs. It:

1. Reads `Authorization: Bearer <token>`
2. Calls the user-service `/v1/tokens/verify` endpoint over HTTP
3. Attaches the verified principal to `req.principal`
4. Returns RFC 7807 401 on missing/invalid token; 503 if user-service unreachable

Phase 1 will swap the remote HTTP call for a local SPIFFE-verified SVID check; the middleware shape stays the same.

```ts
// From packages/auth/src/index.ts
declare module 'fastify' {
  interface FastifyRequest {
    principal?: Principal | null;
  }
}

export async function installAuthPlugin(app, opts) {
  app.addHook('onRequest', async (req, reply) => {
    if (publicPrefixes.some(p => req.url.startsWith(p))) {
      req.principal = null;
      return;
    }
    const auth = req.headers.authorization;
    if (!auth?.startsWith('Bearer ')) { /* 401 */ }
    const principal = await verify(opts.userServiceUrl, token, ...);
    req.principal = principal;
  });
}
```

## 4.7 Teal + amber palette (PPE-aware)

The master plan resolved a cross-agent conflict (Appendix L.1): the Design agent's `cyan + copper` STRATUM palette lost to the Frontend agent's `teal + amber` argument. Cool greens read as "OK / safe / go" in proximity to PPE amber/red on real sites; amber is reserved for warning states only.

```ts
// From packages/tokens/src/index.ts
export const brand: BrandColor = {
  signal500: '#00B894',  // teal — primary action, "go"
  signal300: '#4FDDB6',  // lighter teal for high-contrast on dark
  amber400:  '#F5B544',  // warning ONLY — never used as primary
  amber500:  '#F5A524',
  // ...
} as const;
```

Three modes ship: `light`, `dark` (default), `high-contrast`. The dashboard's `<html data-mode="dark">` emits CSS variables consumed by every component.

### 4.8 Strict TypeScript

`tsconfig.base.json` enables:

```
{
  "strict": true,
  "noUncheckedIndexedAccess": true,
  "exactOptionalPropertyTypes": true,
  "noImplicitOverride": true,
  "noFallthroughCasesInSwitch": true,
  "esModuleInterop": true,
  "skipLibCheck": true,
  "forceConsistentCasingInFileNames": true
}
```

These caught real bugs during development (e.g., the gray-850 token needed for dark-mode "raised" surfaces was missing from the scale; the `PostgresOrgRepository` row-mapping required an index signature; the project update SQL had a 5-parameter signature that the FakePgClient had to match).

## 5. Test Suite & Verification

### 5.1 TDD discipline followed

Every production code change started with a failing test (RED), then the minimal implementation (GREEN), then refactor. The repository contains evidence of the cycle in commits — e.g., the project Postgres repo went through:

1. RED: `postgres-repo.test.ts` written, no implementation → test fails
2. GREEN: `postgres-repo.ts` written → tests pass
3. REFACTOR: index-signature constraints fixed, FakePgClient updated to match the 5-parameter UPDATE SQL

### 5.2 Test counts

```
packages/tokens              11/11 passing
packages/observability        5/5  passing
packages/auth                 7/7  passing
```

```
services/org-service         26/26 passing
services/project-service     25/25 passing
services/user-service        15/15 passing
services/membership-service  19/19 passing
apps/dashboard                6/6  passing
──────────────────────────────────────────────
TOTAL                        114/114 passing (0 failures, 0 skipped)
```

Run command from repo root: `pnpm test`

## 5.3 Build verification

```
packages/tokens          tsc -p tsconfig.json    →  emits dist/
packages/observability   tsc -p tsconfig.json    →  emits dist/
packages/auth            tsc -p tsconfig.json    →  emits dist/
services/org-service     tsc -p tsconfig.json    →  emits dist/
services/project-service tsc -p tsconfig.json    →  emits dist/
services/user-service    tsc -p tsconfig.json    →  emits dist/
services/membership-service tsc -p tsconfig.json → emits dist/
apps/dashboard           next build              →  emits .next/
──────────────────────────────────────────────────────────────
ALL CLEAN
```

Run command: `pnpm build`

## 5.4 Lint

`pnpm lint` runs ESLint 9 + typescript-eslint on the workspace. The current configuration enforces `no-explicit-any` as a warning (relaxed in test helpers) and `consistent-type-imports` as an error.

# 6. Boot Smoke Test Results

End-to-end smoke testing against running services. All commands executed during this session, output captured live.

## 6.1 Health endpoint

```
$ curl -s http://127.0.0.1:9080/v1/health
{"status":"ok"}
```

## 6.2 org-service: create + duplicate + list

```
$ # Create
$ curl -X POST http://127.0.0.1:9081/v1/orgs \
       -H "content-type: application/json" \
       -H "idempotency-key: smoke-1" \
       -d '{"name":"Hudson Tower GC","region":"us-east","plan":"pro"}'
{"id":"org_00000001","name":"Hudson Tower GC","region":"us-east",
 "plan":"pro","createdAt":"2026-08-08T05:04:07.840Z"}

$ # List (1 result)
$ curl http://127.0.0.1:9081/v1/orgs
{"data":[{"id":"org_00000001","name":"Hudson Tower GC",...}]}

$ # Region filter
$ curl 'http://127.0.0.1:9081/v1/orgs?region=eu-west'
{"data":[]}
```

## 6.3 project-service: create + list-by-org

```
$ curl -X POST http://127.0.0.1:9082/v1/projects \
       -H "content-type: application/json" \
       -d '{"orgId":"org_00000001","name":"Hudson
        Tower","address":"500 W 33rd St","startedAt":"2026-01-
        15T00:00:00.000Z"}'
{"id":"prj_00000001","orgId":"org_00000001","name":"Hudson Tower",
 "status":"active",...}

$ curl 'http://127.0.0.1:9082/v1/projects?orgId=org_00000001'
{"data":[{"id":"prj_00000001","orgId":"org_00000001","name":"Hudson
        Tower",...}]}
```

## 6.4 membership-service: auth wired end-to-end

```
$ # Health (public — no auth required)
$ curl http://127.0.0.1:9086/v1/health
{"status":"ok"}                                  HTTP 200

$ # List members without bearer → 401
$ curl http://127.0.0.1:9086/v1/orgs/org_1/members
{"type":"https://sthyra-crm.dev/errors/unauthorized",
 "title":"Missing bearer token",
 "status":401,
 "detail":"Authorization: Bearer *** header is required",
 "trace_id":"496d8981-769c-4602-83c3-5f750a8dd100",
 "code":"unauthorized"}                          HTTP 401

$ # List with bogus bearer → 401
$ curl -H "Authorization: Bearer opaque:fake" \
       http://127.0.0.1:9086/v1/orgs/org_1/members
{"type":"https://sthyra-crm.dev/errors/unauthorized",
 "title":"Invalid or expired token",
 "status":401,
 "trace_id":"44bf705e-4890-45ce-a4ad-9e6018403128",
 "code":"unauthorized"}                          HTTP 401
```

## 6.5 request-id propagation

```
$ curl -i http://127.0.0.1:9080/v1/health -H "x-request-id:
        req_ping_123"
HTTP/1.1 200 OK
x-request-id: req_ping_123     ← echoed by the server
content-type: application/json
{"status":"ok"}
```

Server-side log emitted with the same `request_id`:

```
{"ts":"2026-08-08T05:18:05.201Z","level":"info","service":"sthyra-crm-
        service",
 "request_id":"req_ping_123","msg":"http_request",
 "fields":
        {"method":"GET","url":"/v1/health","status":200,"duration_ms":1}}
```

# 7. Working Agreement

Embedded in the repository's `README.md`. The full text follows the principles below; key invariants are extracted here.

## 7.1 TDD is non-negotiable

```
NO PRODUCTION CODE WITHOUT A FAILING TEST FIRST.
```

The cycle: RED (write failing test) → GREEN (minimal code to pass) → REFACTOR (clean up). If a test passes immediately, you are testing the wrong thing.

## 7.2 Architectural decisions are pinned

| Decision | Don't re-decide |
|---|---|
| Tenancy is data-modeled | Every record carries `region`; (name, region) is unique |
| Repository pattern | InMemory for tests, Postgres for prod. Interface in the service file |
| REST + RFC 7807 + Idempotency-Key | At the edge. Use the existing helpers |
| Request-ID propagation | Via `@sthyra-crm/observability`. Every log has `request_id` |
| Strict TypeScript | `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`, `noImplicitOverride` |
| Teal + amber palette | NOT cyan + copper. NEVER safety-orange-as-primary |

## 7.3 Repository layout

```
packages/<lib>/        # reusable library code (tokens, observability, auth)
services/<svc>/        # stateful backend service (org, project, user, ...)
```

```
apps/<app>/            # end-user surface (dashboard, future mobile-shell)
```

## 7.4 Naming

| Item | Convention |
|---|---|
| Files | `kebab-case.ts` |
| Classes | `PascalCase` |
| Functions / variables | `camelCase` |
| Test files | `*.test.ts` co-located with source |
| DB tables | `snake_case` (e.g., `orgs`, `project_members`) |
| API endpoints | `/v1/<resource>` REST |
| Error responses | `application/problem+json` per RFC 7807 |

## 7.5 Database

- Parameterized queries only. No string concatenation.
- New table = new migration. Idempotent (`CREATE TABLE IF NOT EXISTS`).
- Add an index for any column you query by.

## 7.6 Logging

- Never `console.log`. Use `emit('info' | 'warn' | 'error', msg, fields)` from `@sthyra-crm/observability`.
- Logs are JSON, one line per event.
- Don't log PII, full tokens, or full request bodies.

## 7.7 Common tasks

**Add an endpoint:** 1. Write the failing test first → run it → add the route in `http.ts` using `buildHeaders`/`parseProblem` helpers → add a structured log via `emit()` → commit.

**Add a new service:** Copy `services/org-service/` structure. Service → Repository interface → InMemory impl → Postgres impl (parameterized SQL only) → HTTP → CLI. Tests co-located.

**Add a design token:** Edit `packages/tokens/src/index.ts`. Add to `SemanticColor` interface + all 3 modes (light/dark/high-contrast). Add a test case.

### 7.8 Commit format

`<scope>: <imperative summary>` — `feat(org-service): add Postgres repository.`

## 8. How To Run It

### 8.1 First time setup

```
# Clone or cd into the repo
cd ~/projects/sthyra-crm

# Install (pnpm 11, Node 22)
pnpm install

# Start local Postgres
docker compose up -d postgres

# Build all packages
pnpm build
```

### 8.2 Run tests

```
# Run all tests
pnpm test

# Run tests for a specific package
cd services/org-service && pnpm test
cd packages/tokens && pnpm test
cd apps/dashboard && pnpm test
```

## 8.3 Boot the services

```
# Terminal 1: org-service against Postgres
DATABASE_URL=postgres://sthyra-crm:sthyra-crm@localhost:5432/sthyra-
        crm \
  pnpm --filter=@sthyra-crm/org-service start:pg

# Terminal 2: org-service against in-memory (faster, for demos)
pnpm --filter=@sthyra-crm/org-service start:inmem

# Terminal 3: project-service
pnpm --filter=@sthyra-crm/project-service start:inmem

# Terminal 4: user-service
pnpm --filter=@sthyra-crm/user-service start:inmem

# Terminal 5: membership-service
pnpm --filter=@sthyra-crm/membership-service start:inmem

# Terminal 6: dashboard
pnpm --filter=@sthyra-crm/dashboard dev
# → http://localhost:3000
```

## 8.4 Exercise it

```
# Create an org
curl -X POST http://localhost:8080/v1/orgs \
  -H "content-type: application/json" \
  -H "idempotency-key: demo-1" \
  -d '{"name":"Hudson Tower GC","region":"us-east","plan":"pro"}'

# Create a project under that org
curl -X POST http://localhost:8082/v1/projects \
  -H "content-type: application/json" \
  -d '{"orgId":"org_00000001","name":"Hudson Tower","address":"500 W
        33rd St","startedAt":"2026-01-15"}'

# Open the dashboard
open http://localhost:3000
```

## 8.5 Run the CI pipeline locally

```
# Equivalent to what GitHub Actions runs:
pnpm test
pnpm build
```

The Postgres integration step requires Docker and is only exercised in the GitHub Actions environment, but you can simulate it locally by:

```
docker compose up -d postgres
DATABASE_URL=postgres://sthyra-crm:sthyra-crm@localhost:5432/sthyra-
        crm \
  pnpm --filter=@sthyra-crm/org-service start:pg &
SERVER_PID=$!
sleep 2
curl -X POST http://127.0.0.1:8080/v1/orgs \
  -H "content-type: application/json" \
  -d '{"name":"CI Test Org","region":"us-east","plan":"pro"}' | grep
        '"id":"org_'
kill $SERVER_PID
echo "POSTGRES_ORG_REPO_OK"
```

# 9. Honest Scope Statement

This section is critical. The Phase 0 Foundation ships a real, working, tested backend. The full product plan specifies 13 products, an 18-month roadmap, ~$8.4M Year-1 budget, 12 founding hires, GPU fleet, FedRAMP authorization, native mobile apps, AI Copilot, and 20+ integrations. **None of that ships in Phase 0.** Anything that would have been fabricated instead of built is documented here as a deliberate non-goal.

## 9.1 What was NOT built

| Feature | Why not | Phase |
|---|---|---|
| 360° viewer (three.js, WebGL/WebGPU) | Requires senior 3D engineer + WebGL pipeline | Phase 1 |

| Feature | Why not | Phase |
|---|---|---|
| Capture pipeline (Spatial AI: COLMAP/GLOMAP, OpenMVS, 3D Gaussian Splatting, SAM-2, DINOv2) | Requires ML infra + GPU fleet | Phase 1 |
| AI Copilot (Llama 3, RAG, function-calling, voice I/O) | Requires LLM infra + safety pipeline + citations | Phase 1 |
| Mobile apps (KMM + Swift/Kotlin shells, ARKit/ARCore) | Requires native mobile engineers + hardware partnerships | Phase 1 |
| Real OIDC / SAML SSO / SCIM | The user-service has a stubbed interface (`0paque:hex` tokens); real OIDC requires Auth0/Okta/Entra integration | Phase 1 |
| SPIFFE/SPIRE workload identity | Decided in plan (ADR-5); requires SPIRE deployment | Phase 1 |
| Terraform / Kubernetes / multi-region | Requires DevOps/SRE handoff | Phase 2 |
| Integrations (Procore, ACC, BIM 360, P6, Salesforce, ServiceNow, Slack, Teams, Box, GDrive, DocuSign, Smartsheet, Bluebeam, Aconex, Outlook, GCal — 16+) | Each is its own service | Phase 1–2 |
| FedRAMP / SOC 2 / ISO 27001 evidence collection | Requires audit handoff | Phase 2 |
| Phase-3 GA features (Sthyra CRM Live multi-stakeholder walkthroughs, Sthyra | Each is its own product track | Phase 4 |

| Feature | Why not | Phase |
|---|---|---|
| CRM Twin FM integrations, Sthyra CRM ESG, Sthyra CRM Claims, Drone-in-a-Box) | | |

## 9.2 What WOULD have been fabricated if we cut corners

Each item above could have been "built" with stub code that compiles but doesn't actually work. Examples of what we deliberately did **not** do:

- **No fake 360 viewer.** A `THREE.js` skeleton with a placeholder sphere would have been misleading.
- **No fake OIDC.** A mock auth screen that returns a hardcoded JWT would hide the integration work the team needs to do.
- **No fake "AI Copilot".** A button that returns `echo "I'm an AI"` would have been worse than nothing.
- **No fake "ML pipeline".** An empty `pipeline.py` with `def run(): pass` would have hidden the real Phase 1 work.

The working agreement in §7 explicitly forbids this:

> If a task seems to require a Phase 1+ feature, **don't fabricate a half-working version**. Surface it as a Phase 1+ item in your final report and stop.

# 10. Detailed Steps To Be Done (Phase 1+ Backlog)

This is the implementation roadmap for everything that did not fit in Phase 0. Each item includes exact file paths, code shape, test commands, and verification steps. Where useful, sample code or migrations are provided inline.

The roadmap is structured by **dependency order**, not by master-plan phase. Items lower in the dependency graph come first.

# 10.1 Phase 1.A — Auth & Identity (auth, unblocks ABAC everywhere)

## 10.1.1 Real OIDC provider integration

**Why:** Today `verifyToken()` in user-service issues opaque `opaque:hex` tokens. Real OIDC requires integrating Auth0/Okta/Entra so users federate through their employer's identity provider.

**Files to change:** - `services/user-service/src/index.ts` — extend `UserService.provision()` to accept an `oidcSubject` parameter; remove password handling. - `services/user-service/src/http.ts` — replace `POST /v1/users` with `POST /v1/users/oidc-callback` that maps `oidcSubject → userId`. - New: `services/user-service/src/oidc.ts` — provider-specific config. - `packages/auth/src/index.ts` — replace HTTP verify with local JWT signature verification (JWKS-based, with cached keys).

**Code shape (oidc.ts):**

```
import { Issuer, Client, generators } from 'openid-client';

export async function buildOidcClient(issuerUrl: string, clientId:
        string, clientSecret: string) {
  const issuer = await Issuer.discover(issuerUrl);
  return new issuer.Client({ client_id: clientId, client_secret:
        clientSecret });
}

export async function exchangeCode(client: Client, code: string,
        redirectUri: string) {
  const params = client.callbackParams({ code, iss: '' /* filled by
        provider */ });
  return client.callback(redirectUri, params);
}
```

**Test plan:** - Unit test for `buildOidcClient` (mocked `Issuer.discover` - Integration test against Auth0 dev tenant (env vars required) - E2E: provider login → token → dashboard renders user

**Verification:** A user can log in via the OIDC provider and the resulting JWT is verified locally by `@sthyra-crm/auth` without an HTTP round-trip.

### 10.1.2 SAML SSO

**Why:** Enterprise customers (especially FedRAMP) require SAML SSO. Auth0/Okta abstract this, but we need the config UI and the SAML-metadata exchange.

**Files to change:** - New: `services/user-service/src/saml.ts` - `services/user-service/src/http.ts` — `POST /v1/orgs/:orgId/saml/config` - `apps/dashboard/src/app/orgs/[orgId]/sso/page.tsx` — admin UI

**Verification:** A customer can upload their IdP's SAML metadata XML and provisioned users can log in via SAML.

### 10.1.3 SCIM provisioning

**Why:** Enterprise customers want their HR system to provision/deprovision users automatically.

**Files to change:** - New: `services/user-service/src/scim.ts` — SCIM 2.0 endpoints - `/scim/v2/Users`, `/scim/v2/Groups` per RFC 7644 - Hooks into the user-service provision/deprovision path

**Verification:** A SCIM client (Okta, Azure AD) can create users in Sthyra CRM via SCIM, and deprovisioning from the IdP removes access within 5 minutes.

### 10.1.4 MFA / step-up auth

**Why:** Master plan §2.2 specifies FIDO2/WebAuthn for admins, TOTP for field BYOD. The security agent's concrete policy table (Appendix I) requires step-up MFA on sensitive operations.

**Files to change:** - `services/user-service/src/mfa.ts` — TOTP enrollment, FIDO2 challenge - `services/user-service/src/http.ts` — `POST /v1/mfa/totp/enroll`, `POST /v1/mfa/totp/verify`, `POST /v1/mfa/webauthn/challenge` - `packages/auth/src/index.ts` — surface MFA requirement on `req.principal`

**Verification:** An admin attempting to archive a project without an MFA-valid session gets a 401 with `mfa_required: true` in the response.

### 10.1.5 SPIFFE/SPIRE workload identity

**Why:** Master plan ADR-5. Service-to-service mTLS with SVIDs (TTL ≤ 1h, auto-rotated).

**Files to change:** - `packages/auth/src/index.ts` — accept SVIDs as an alternative to bearer tokens; verify against local SPIRE agent's JWKS - New: `infra/spire/` — SPIRE server + agent deployment configs - `docker-compose.yml` — add SPIRE for local dev

**Code shape:**

```
// Verify a SPIFFE SVID's signature against the SPIRE trust bundle
import { verifyJWT, createRemoteJWKSet } from 'jose';
const jwks = createRemoteJWKSet(new URL('https://spire-
        agent:8443/.well-known/jwks.json'));
const { payload } = await verifyJWT(svid, jwks);
return { userId: payload.sub, orgId: payload['x-sthyra-crm-org'],
        role: payload['x-sthyra-crm-role'] };
```

**Verification:** Two services can call each other over mTLS with auto-rotated SVIDs; no bearer tokens in service-to-service traffic.

## 10.2 Phase 1.B — Postgres parity for remaining services

### 10.2.1 User-service Postgres repository

**Why:** Today user-service has only an in-memory store. Real users can't be served from memory.

**Files to change:** - New: `services/user-service/src/postgres-repo.ts` — same pattern as `org-service/src/postgres-repo.ts` - New: `services/user-service/src/postgres-cli.ts` - Update `services/user-service/src/http.ts` to accept the Postgres repo via DI

**Code shape (sketch):**

```
CREATE TABLE IF NOT EXISTS users (
  id           TEXT PRIMARY KEY,
  email        TEXT NOT NULL,
  display_name TEXT NOT NULL,
```

```
  role        TEXT NOT NULL,
  org_id      TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (org_id, LOWER(email))
);
CREATE TABLE IF NOT EXISTS tokens (
  token_hash  TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL REFERENCES users(id),
  org_id      TEXT NOT NULL,
  role        TEXT NOT NULL,
  issued_at   TIMESTAMPTZ NOT NULL,
  expires_at  TIMESTAMPTZ NOT NULL
);
CREATE INDEX IF NOT EXISTS tokens_expires_at_idx ON tokens
        (expires_at);
```

**Test plan:** - Mirror `org-service/src/postgres-repo.test.ts`: - insert + findById round-trip - findByEmail case-insensitive via `LOWER()` - `UniqueViolationError` on `(orgId, email)` duplicate - parameterized SQL only (no string concat) - migration idempotency

**Verification:** `pnpm --filter=@sthyra-crm/user-service start:pg` boots against Postgres, `POST /v1/users` and `POST /v1/users/:id/tokens` work end-to-end.

### 10.2.2 Membership-service Postgres repository

Same pattern. The current `InMemoryMembershipRepository` has these methods to implement:

```
interface MembershipRepository {
  insertOrgMember(member: OrgMembership): Promise<void>;
  insertProjectMember(member: ProjectMembership): Promise<void>;
  findOrgMember(userId: string, orgId: string): Promise<OrgMembership
        | null>;
  findProjectMember(userId: string, projectId: string):
        Promise<ProjectMembership | null>;
  upsertProjectMember(member: ProjectMembership): Promise<void>;
  listOrgMembers(orgId: string): Promise<OrgMembership[]>;
  listProjectMembers(projectId: string): Promise<ProjectMembership[]>;
  listProjectsForUser(userId: string): Promise<ProjectMembership[]>;
  deleteOrgMember(userId: string, orgId: string): Promise<void>;
}
```

The SQL would include:

```
CREATE TABLE IF NOT EXISTS org_memberships (
  id          TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL REFERENCES users(id),
  org_id      TEXT NOT NULL REFERENCES orgs(id),
  role        TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (user_id, org_id)
);
CREATE TABLE IF NOT EXISTS project_memberships (
  id          TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL REFERENCES users(id),
  project_id  TEXT NOT NULL REFERENCES projects(id),
  role        TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (user_id, project_id)
);
```

**Verification:** `pnpm --filter=@sthyra-crm/membership-service start:pg` boots against Postgres.

# 10.3 Phase 1.C — Capture service foundation

## 10.3.1 Capture ingestion endpoint (stubbed spatial AI)

**Why:** Without a capture service, the project cannot ingest 360° video walks from the mobile app.

**Files to create:** - New service: `services/capture-service/` (mirror `services/org-service/`) - `services/capture-service/src/index.ts` — CaptureService + Repository - `services/capture-service/src/http.ts` — POST /v1/captures, upload-session endpoint, finalize - `services/capture-service/src/postgres-repo.ts` - `services/capture-service/src/storage.ts` — S3-compatible blob storage abstraction (LocalFsStorage for dev, S3Storage for prod)

**Endpoint shape:**

```
POST /v1/projects/:projectId/captures
Content-Type: application/json
```

```
Idempotency-Key: req-abc

{
  "clientCaptureId": "uuid-from-mobile",
  "kind": "walkthrough_360",
  "startedAt": "2026-08-08T14:00:00Z",
  "device": { "model": "Insta360 X4", "osVersion": "iOS 17.5" }
}
→ 201
{
  "id": "cap_00000001",
  "uploadSession": { "id": "upl_xxx", "chunkUrls": [...] }
}

PUT /v1/upload-sessions/:id/chunks/:index
Content-MD5: base64-md5
Body: <binary>
→ 200 { "received": true, "etag": "..." }
```

**Test plan:** - CaptureService.create returns id + upload session - Upload chunk idempotency (re-upload of same chunk returns 200 not 409) - Finalize triggers the pipeline (stub: just writes a `pipeline_runs` row in `processing` state) - Redis-backed idempotency cache

**Verification:** A test capture can be ingested, marked finalized, and a `pipeline_runs` row exists.

### 10.3.2 Pipeline orchestrator stub

**Why:** We need a job queue abstraction before we can plug in real COLMAP/GLOMAP.

**Files to create:** - `services/capture-service/src/pipeline.ts` — `enqueueRun`, `getRunStatus`, `cancelRun` - `services/capture-service/src/pipeline-queue.ts` — Redis Streams-based queue with at-least-once delivery + DLQ

**Verification:** A finalized capture produces a `pipeline_run` row, the queue consumer picks it up (in a stubbed test mode), and the run transitions through stages: `pending → decoding → sfm → meshing → done`.

### 10.3.3 LocalFsStorage

**Why:** Without S3, dev environments can't store anything.

**Files:** - `services/capture-service/src/storage/local-fs.ts` — implementation - `services/capture-service/src/storage/s3.ts` — implementation (interface- identical)

```
export interface BlobStorage {
  put(key: string, body: Buffer, contentType: string): Promise<void>;
  get(key: string): Promise<Buffer>;
  head(key: string): Promise<{ size: number; contentType: string }>;
  signedUrl(key: string, ttlSeconds: number): Promise<string>;
}
```

**Verification:** Capture chunks written to `LocalFsStorage` round-trip through `get`.

## 10.4 Phase 1.D — Mobile apps (the biggest Phase 1 work item)

### 10.4.1 KMM (Kotlin Multiplatform) shared core

**Why:** Per master plan §6.2, mobile shells share ~75% of logic.

**Files:** - New: `apps/mobile-kmm/` — Kotlin Multiplatform module - `apps/mobile-kmm/src/commonMain/kotlin/com/sthyra-crm/sync/` — sync engine - `apps/mobile-kmm/src/commonMain/kotlin/com/sthyra-crm/models/` — domain types - `apps/mobile-kmm/src/commonMain/kotlin/com/sthyra-crm/crypto/` — Blake3 + AES-GCM

**Code shape (sync engine):**

```
class SyncEngine(
    private val localStore: LocalStore,
    private val remote: RemoteApi,
    private val clock: Clock = Clock.System,
) {
    suspend fun sync(): SyncResult {
        val dirty = localStore.dirtyRecords()
        for (record in dirty) {
            remote.upload(record).fold(
```

```
                onSuccess = { localStore.markSynced(record.id,
        it.serverVersion) },
                onFailure = { localStore.markFailed(record.id,
        it.error) }
            )
        }
        val updates = remote.fetchSince(localStore.lastSyncCursor())
        for (u in updates) localStore.upsert(u)
        return SyncResult(uploaded = dirty.size, downloaded =
        updates.size)
    }
}
```

**Test plan:** Unit tests on JVM (`commonTest`) for the sync state machine; integration tests against a mocked remote.

**Verification:** `gradle :mobile-kmm:jvmTest` passes.

### 10.4.2 iOS native shell

**Why:** Capture preview at 30fps with IMU fusion requires Metal + AVFoundation.

**Files:** - New: `apps/mobile-ios/` — Xcode project - `apps/mobile-ios/Capture/` — AVCaptureSession + CoreMotion pipeline - `apps/mobile-ios/Pairing/` — CoreBluetooth camera pairing - `apps/mobile-ios/Sync/` — KMM-generated Swift bindings

**Test plan:** XCUITest for the capture flow; unit tests for IMU fusion.

**Verification:** A field test on a real device captures a 360° walk that uploads via the KMM sync engine.

### 10.4.3 Android native shell

Mirror of iOS shell, using Camera2 + SensorManager + WorkManager.

### 10.4.4 On-device Whisper for dictation

**Files:** - New: `apps/mobile-kmm/src/commonMain/kotlin/com/sthyra-crm/asr/` — Whisper.cpp KMP binding - Model asset delivered on first launch (~75 MB for `tiny`, ~150 MB for `base`)

**Verification:** Dictation in the field produces a transcript offline.

### 10.4.5 Indoor positioning

**Files:** - `apps/mobile-kmm/src/commonMain/kotlin/com/sthyra-crm/indoor/` — particle filter over floor plan + Wi-Fi RTT + BLE beacons - Calibration flow in iOS/Android shells

**Verification:** Walking between rooms updates the `FloorAreaEstimate` in the field notes composer.

## 10.5 Phase 1.E — AI Copilot

### 10.5.1 Copilot gateway service

**Files:** - New: `services/copilot-service/` - `services/copilot-service/src/index.ts` — function-calling LLM agent - `services/copilot-service/src/tools/` — `queryCaptures`, `spatialQuery`, `diffCaptures`, `createIssue`, `generateReport`, `draftRfi`, `summarizeProgress`, `measure` - `services/copilot-service/src/safety.ts` — pre/post inference safety pipeline (PII redactor → prompt firewall → context quarantine → inference → content classifier → PII inverse → C2PA provenance)

**Verification:** A user query like “show me open RFIs on Level 3 over $5k” returns a list with citations.

### 10.5.2 Vector store integration

**Files:** - `services/copilot-service/src/rag.ts` — pgvector (or Pinecone/Weaviate) embeddings store - Embeddings: BGE-M3 for text/code, OpenCLIP for visual - Indexing pipeline that runs on capture finalize

**Verification:** A copilot query with `cite` requirement resolves to a specific capture frame, BIM element, or RFI.

### 10.5.3 Voice I/O

**Files:** - `services/copilot-service/src/asr.ts` — Whisper-large-v3-turbo via faster-whisper - `services/copilot-service/src/tts.ts` — Orpheus-1B or ElevenLabs v3 - Barge-in via WebRTC AEC pipeline

**Verification:** A user can speak to the copilot and hear the response.

# 10.6 Phase 1.F — 360° viewer (web)

## 10.6.1 WebGL viewer with three.js

**Files:** - `apps/dashboard/src/components/viewer-360/` — three.js + @react-three/fiber - Equirectangular + cube-map projections - WebGPU primary, WebGL2 fallback - LOD downshift on low-end GPUs

**Verification:** A capture with 8K equirectangular frames renders at 60fps on M1 MacBook, 45fps on Intel Iris Xe, with LOD downshift on integrated graphics.

## 10.6.2 BIM overlay

**Files:** - `apps/dashboard/src/components/bim-viewer/` — three.js glTF + custom IFC loader (web-ifc) + NWD via Autodesk Forge Data Exchange - Section plane, measurement, BCF 3.0 import/export

**Verification:** A user can drop a BIM model into the viewer and click elements to see their capture-vs-model deviation.

## 10.6.3 Split View + Reveal slider

**Files:** - `apps/dashboard/src/components/split-view/` — synchronized two-pane (capture | BIM) with shared camera + reveal wipe

**Verification:** Dragging the reveal slider transitions between as-built and as-designed in real time.

## 10.6.4 `/immersive` marketing page (the full one)

**Files:** - `apps/marketing/` — new Next.js app with cinematic 360 entry path - 14-second camera path through a curated walkthrough - Time scrub through 6 months of captures - Inspector mode with photo evidence, BIM diff, trade attribution, ETA - WebGL2 fallback to static panorama - `prefers-reduced-motion` fallback (timeline scrub only)

**Verification:** A new visitor can complete the `/immersive` walkthrough on Chrome, Safari, Firefox, and mobile browsers, with and without reduced motion.

# 10.7 Phase 2 — Infrastructure & integrations

## 10.7.1 Terraform / Kubernetes / multi-region

**Files:** - New: `infra/terraform/` — VPC, EKS, RDS, ElastiCache, S3, CloudFront per region - New: `infra/k8s/` — Helm charts for each service - New: `infra/argocd/` — ApplicationSets per region

**Verification:** `terraform plan` produces a clean diff; `argocd app sync` deploys all services to a fresh region.

## 10.7.2 Integrations (priority order from PRD)

For each integration, the pattern is the same:

```
services/integration-<vendor>/
  src/index.ts          # Integration service
  src/webhook-handler.ts # Receives vendor webhooks
  src/sync.ts           # Pull-based sync on a schedule
  src/transform.ts      # Vendor → Sthyra CRM type mapping
  src/*.test.ts         # TDD
```

| Integration | Phase | Notes |
|---|---|---|
| Procore | 1 | Most-requested by GCs |
| Autodesk Construction Cloud / BIM 360 | 1 | BIM model source |
| Oracle Primavera P6 | 1 | Schedule source |
| Microsoft Project | 2 | Schedule (P6 lite) |
| Salesforce | 2 | CRM/owner data |
| ServiceNow | 2 | Issue escalation |
| Slack | 2 | Notifications |
| Microsoft Teams | 2 | Notifications |
| Box / Google Drive | 2 | Document storage |
| DocuSign | 2 | Lien waivers |
| Smartsheet | 2 | Subcontractor scheduling |
| Bluebeam | 2 | Markup annotations |

| Integration | Phase | Notes |
|---|---|---|
| Aconex | 2 | Owner-side document control |
| Outlook / Google Calendar | 2 | OAC meeting scheduling |

**Verification:** A test tenant in Procore + ACC + P6 round-trips: create a project in Sthyra CRM → see it in Procore; create an RFI in Procore → see it in Sthyra CRM.

### 10.7.3 FedRAMP Moderate authorization

**Files:** - `infra/spire/` (already in 10.1.5) - `infra/fedramp/` — SSP (System Security Plan), POA&M (Plan of Action & Milestones), ConMon (Continuous Monitoring) scripts - New: `services/*/audit-log.ts` — every state-changing action emits to append-only audit log

**Timeline:** 12–18 months. StateRAMP is a faster interim.

### 10.7.4 SOC 2 Type II audit

**Files:** - `compliance/soc2/` — controls matrix, evidence collection, audit scripts

**Timeline:** Day-one of Phase 1. Type II report in 6–12 months.

### 10.7.5 ISO 27001

**Files:** - `compliance/iso27001/` — ISMS (Information Security Management System)

**Timeline:** Initiated Phase 2.

## 10.8 Phase 3 — Differentiator products

### 10.8.1 Sthyra CRM Live (multi-stakeholder walkthroughs)

Multi-user presence in the 360 viewer with pass-the-pointer and auto-minutes. WebRTC SFU + presence via Redis Pub/Sub + recordings stored to S3.

### 10.8.2 Sthyra CRM Twin (digital twin + handover)

Continuous delta detection on every capture + handover package (federated model + capture history + O&M + ESG) + FM integrations (Akila, Willow, AWS IoT TwinMaker, Azure Digital Twins).

### 10.8.3 Sthyra CRM ESG

Embodied carbon tracking (BIM material quantities → CO2e), waste tracking, LEED/Envision/BREEAM credit readiness, GRESB/MSCI/SEC climate disclosure.

### 10.8.4 Sthyra CRM Claims (legal-grade)

RFC 3161 timestamps, notarization, dispute-ready single-PDF export, expert-witness export template.

### 10.8.5 Drone-in-a-Box

DJI Dock + Skydio Dock integration. Scheduled autonomous flights. BVLOS-aware flight logs. LAANC integration where available.

### 10.8.6 On-prem / air-gapped deployment

DOD/intel/nuclear customers require fully air-gapped deployments. The `Sthyra CRM Twin` and `Sthyra CRM Claims` products are the wedge into this market.

### 10.8.7 Regional AI Copilot inference

EU, JP, KSA data residency requires that Copilot inference happens in- region. Llama 3 70B on-prem for defense, Azure OpenAI Service for EU, Sakura Internet for JP.

### 10.8.8 CMMC L3, HIPAA, FedRAMP High

Progression beyond FedRAMP Moderate for defense/healthcare customers.

# 11. Risk Register & Open Questions

These are risks I am flagging based on the work done in Phase 0. Each should be reviewed in Phase 1 planning.

## 11.1 Tech risks

| # | Risk | Severity | Mitigation |
|---|---|---|---|
| R1 | Opaque `opaque:hex` tokens have no standard revocation API. If a user needs to invalidate all their sessions today, we delete by token hash, which only works if we have the hash. | Medium | Phase 1: replace with JWT + JTI denylist in Redis; Phase 2: SPIFFE SVIDs (≤1h TTL, auto-rotated) |
| R2 | `InMemoryOrgRepository` and `InMemoryProjectRepository` use single-process counters. Multi-process deployments will collide. | High | Already addressed by `PostgresOrgRepository` + `PostgresProjectRepository`; remove the in-memory repos in Phase 2 once every service has Postgres parity |
| R3 | The `InMemoryTokenStore` in user-service doesn't survive restart. | High | Phase 1: replace with Redis-backed store; the `TokenStore` interface is already designed for this swap |
| R4 | The auth middleware makes a synchronous HTTP call to user-service on every request. | Medium | Phase 1: cache verified tokens in Redis with short TTL; Phase 2: replace with local JWT/SVID verification |
| R5 | The dashboard uses Next.js App Router with `force-dynamic` on the home page — every request triggers a full SSR data fetch. | Low | Acceptable for Phase 0 traffic. Phase 1: add ISR for the marketing pages, RSC streaming for the dashboard |
| R6 | The `FakePgClient` in tests mimics the SQL surface, not Postgres behavior. Tests passing with `FakePgClient` do not prove Postgres will work. | High | CI includes a Postgres service container (already configured); Phase 1 must run the postgres-repo tests against the real DB, not just the fake |

| # | Risk | Severity | Mitigation |
|---|---|---|---|
| R7 | `noUncheckedIndexedAccess` is strict; we don't use lodash-style `??` defaults everywhere. | Low | Lint rule: prefer `??` `defaultValue` to `!` assertions |

## 11.2 Product risks

| # | Risk | Severity | Mitigation |
|---|---|---|---|
| R8 | The OpenSpace competitor has 5+ years of captures in customer accounts. Switching cost is real. | High | Import tool: geometry + pins + timeline scrub; ship GA-day-one |
| R9 | The 13-product roadmap is large. Trying to ship all of Phase 1 in parallel risks quality. | High | The bite-sized tasks in §10 are designed for one-engineer-per-product. Hire the team per product, not per role |
| R10 | Pricing: $480/seat/mo with per-capture credits needs validation. $10K minimum for sub-$40M-revenue customers (OpenSpace's pattern) is a friction point. | Medium | Run a pricing study with 10 design partners before GA |
| R11 | Drone regulatory complexity (FAA Part 107, BVLOS | Medium | Partner with certified operators; |

| # | Risk | Severity | Mitigation |
|---|---|---|---|
| | waivers, EU divided skies). | | consider leasing drones-as-a-service rather than operating our own |

## 11.3 Open questions for the founding team

These surfaced during planning but were not resolved:

1. **Pricing model.** Seat + storage + AI credits? Per-project + per-SF? Trade-first freemium? Need pricing study with 10 design partners.
2. **Trades strategy.** Sub-first (under OpenSpace's radar) or GC-first?
3. **On-prem / air-gapped.** How much of the product supports full air-gapped for DOD, intel community, nuclear?
4. **OpenSpace/StructionSite import fidelity.** Old captures are the switching cost. How deep do we go — geometry + pins + scrub, or whole-account migration?
5. **BIM dependency.** If a project has no model, do we shut down Sthyra CRM Model or generate a base model from the floor plan?
6. **Hardware revenue.** Sthyra CRM-branded 360 cameras / drones? Neutral is safer long-term; branded is higher-margin short-term.
7. **International expansion order.** UK, KSA, India, Brazil, Germany?
8. **Insurance & warranty.** Sthyra CRM Claims opens legal exposure; need product liability + E&O coverage from day one.

# 12. Appendix: Resolved Cross-Agent Conflicts

When the 10 specialist agents designed the system independently, four conflicts surfaced. The master plan resolved all four and the implementation reflects the resolutions.

## L.1 Palette: cyan + copper vs teal + amber

**Resolution: Teal `#00B894` + amber `#F5A524`.**

The Frontend agent's argument won: "Cool greens read as 'OK / safe / go' in proximity to PPE amber/red. The empty trait of safety-orange-as-primary creates confusion with on-site stop signs." Cyan + copper was the prettier deck answer; teal + amber survives a real jobsite.

## L.2 Cost-per-capture

**Resolution: $15 fully-loaded per capture at scale**.

Derived from the CV/3D agent's bottom-up model: compute $13 + storage $0.40 + LLM $0.85 + telemetry $0.25. 35% margin to retail at scale.

## L.3 Time-to-viewable

**Resolution: Two-tier disclosure.**

- First-preview 360 in ≤10 min (low-res tile pyramid + camera pose) → marketing claim
- Fully photoreal novel-view in p50 ≤1.3 h, p95 ≤3 h → architect-facing technical number

## L.4 Brand codename

**Resolution: Sthyra CRM.**

Three agents each invented their own (Sthyra CRM, STRATUM, SiteLens). A sthyra-crm line is the oldest truth-instrument in building — the right name for a product whose value proposition is ground truth. STRATUM becomes the internal design-system layer name; SiteLens is dropped.

## L.5 Product scope expansion

The async delivery of the Product Lead's PRD added four products the master plan hadn't initially included:

- **Sthyra CRM Voice** — hands-free “Hey Sthyra CRM” + on-device Whisper
- **Sthyra CRM Live** — multi-stakeholder live walkthroughs
- **Sthyra CRM Edge** — on-device AI on phones/tablets
- **Sthyra CRM Claims** — legal-grade notarized capture (renamed from “Forensics”)

Master plan §1 was updated to include all 13 products.

# Closing Notes

This document is the handoff packet for the Sthyra CRM Phase 0 Foundation. The master plan, the technical appendix, and the per-package deep-dives from the 10 specialist agents all live in the operator’s plan store at `~/.hermes/plans/`.

**For the next engineer (human or AI):** the README.md in the repository root is the day-one starting point. The working agreement in §7 of this document is enforced by the test suite and the build. The 114 passing tests are your safety net — any change that breaks them needs to be justified or reverted.

**For the next session:** §10 (“Detailed Steps To Be Done”) is the roadmap. Each subsection is a bite-sized TDD task. Pick the highest-leverage item, do it, commit, and update the roadmap.

*End of report.*